

Top 50 ASP.NET Core / .NET Interview Questions (With Short Definitions)

1. What is a class and object in C#?

A class is a blueprint used to create objects. An object is an instance of a class that contains data and behavior.

2. What is a constructor?

A constructor is a special method used to initialize objects. It is automatically called when an object is created.

3. What are access specifiers in C#?

Access specifiers define the visibility of classes and members. Examples include public, private, protected, and internal.

4. What is an interface in .NET?

An interface defines a contract that classes must implement. It contains method declarations without implementation.

5. What is abstraction in C#?

Abstraction hides implementation details and exposes only essential features. It is achieved using abstract classes or interfaces.

6. How can we achieve multiple inheritance in C#?

C# does not support multiple inheritance with classes. It can be achieved using multiple interfaces.

7. What are SOLID principles?

SOLID principles are five design principles that help build maintainable and scalable software. They improve code structure and flexibility.

8. What is the Single Responsibility Principle?

This principle states that a class should have only one responsibility. It makes code easier to maintain and modify.

9. What is the Dependency Inversion Principle?

High-level modules should not depend on low-level modules. Both should depend on abstractions.

10. Difference between Dependency Injection and Dependency Inversion?

Dependency Inversion is a design principle. Dependency Injection is a technique used to implement that principle.

11. What are the main features of ASP.NET Core?

ASP.NET Core is a cross-platform framework for building web applications. It provides high performance and built-in dependency injection.

12. What is middleware in ASP.NET Core?

Middleware are components in the request pipeline. They handle requests and responses in an ASP.NET Core application.

13. Can we create custom middleware?

Yes, developers can create custom middleware. It allows adding custom logic in the request pipeline.

14. Difference between `app.Use()` and `app.Run()`?

`app.Use()` allows the request to pass to the next middleware. `app.Run()` terminates the pipeline and returns a response.

15. Why do we use `Program.cs` in .NET Core?

`Program.cs` is the entry point of the application. It configures services and middleware before running the app.

16. Why do we use `appsettings.json`?

`appsettings.json` stores application configuration settings. It commonly contains connection strings and environment settings.

17. How do we manage configuration in ASP.NET Core?

Configuration can be managed using `appsettings.json`, environment variables, and secret managers. ASP.NET Core provides a built-in configuration system.

18. What are environment variables in ASP.NET Core?

Environment variables store configuration outside the code. They help manage different settings for development, staging, and production.

19. What are HTTP methods?

HTTP methods define actions performed on resources in an API. Common methods include GET, POST, PUT, and DELETE.

20. What are HTTP status codes?

HTTP status codes indicate the result of a request. Examples include 200 (OK), 404 (Not Found), and 500 (Server Error).

21. What is CORS?

CORS allows or restricts requests from different domains. It helps secure APIs when accessed from web applications.

22. What is routing in ASP.NET Core Web API?

Routing maps incoming HTTP requests to controller actions. It determines which endpoint should handle the request.

23. What is attribute routing?

Attribute routing defines routes directly on controller actions using attributes. It provides better control over API endpoints.

24. What is JWT authentication?

JWT is a token-based authentication mechanism. It securely transfers user identity information between client and server.

25. What are the three parts of a JWT token?

A JWT token consists of Header, Payload, and Signature. These parts ensure secure and verifiable communication.

26. How does JWT authentication work?

The server generates a token after user login. The client sends the token with each request for authentication.

27. What is role-based authentication?

Role-based authentication restricts access based on user roles. For example, Admin and User roles may have different permissions.

28. How do we secure ASP.NET Core APIs?

APIs can be secured using authentication, authorization, HTTPS, and token-based security like JWT.

29. What is Dependency Injection?

Dependency Injection is a design pattern used to inject dependencies into classes. It improves testability and loose coupling.

30. How do we implement Dependency Injection in ASP.NET Core?

Services are registered in Program.cs using AddScoped, AddSingleton, or AddTransient. They are injected via constructors.

31. What are DI service lifetimes?

ASP.NET Core supports three lifetimes: Singleton, Scoped, and Transient. These control how long services live.

32. What is Entity Framework Core?

Entity Framework Core is an ORM framework for .NET. It allows developers to work with databases using C# objects.

33. Why do we use DbContext?

DbContext represents a session with the database. It is responsible for querying and saving data.

34. What is DbSet in EF Core?

DbSet represents a table in the database. It allows performing CRUD operations on entities.

35. What is Eager Loading and Lazy Loading?

Eager loading loads related data immediately. Lazy loading loads related data only when it is accessed.

36. Why do we use AsNoTracking()?

AsNoTracking improves performance by disabling change tracking. It is useful for read-only queries.

37. What are Split Queries in EF Core?

Split queries break a complex query into multiple queries. This improves performance when loading large related data.

38. What is LINQ in .NET?

LINQ stands for Language Integrated Query. It allows querying collections and databases using C# syntax.

39. Difference between IEnumerable and IQueryable?

IEnumerable executes queries in memory. IQueryable executes queries at the database level.

40. How do we use LINQ for pagination?

Pagination can be implemented using Skip() and Take() methods. This retrieves a limited number of records.

41. What is DTO?

DTO stands for Data Transfer Object. It is used to transfer data between layers or API and client.

42. What is the Repository Pattern?

Repository pattern separates data access logic from business logic. It improves maintainability and testability.

43. How do we implement global exception handling?

Global exception handling can be implemented using middleware or exception filters. It centralizes error handling.

44. What is asynchronous programming?

Asynchronous programming allows tasks to run without blocking the main thread. It improves performance and scalability.

45. Difference between synchronous and asynchronous programming?

Synchronous execution runs tasks sequentially. Asynchronous execution allows tasks to run independently.

46. Why do we use `async` and `await`?

`async` and `await` simplify asynchronous programming in C#. They allow non-blocking code execution.

47. Why do we use the `using` keyword?

The `using` keyword ensures resources are disposed automatically. It is commonly used with files and database connections.

48. What is microservices architecture?

Microservices architecture divides an application into small independent services. Each service handles a specific business function.

49. What is monolithic architecture?

Monolithic architecture is a single unified application. All components are tightly integrated into one system.

50. Difference between microservices and monolithic architecture?

Microservices split applications into independent services. Monolithic architecture keeps all components in one application.